BÀI THỰC HÀNH MÔN HỌC MÁY – PRACTICE 1

Phân loại email Spam và Ham

Báo cáo thực hành chi tiết: Thiết kế chương trình, phương pháp toán học, kết quả thực nghiệm và phân tích lỗi sai

Bài thực hành xây dựng một chương trình Python hoàn chỉnh nhằm giải quyết bài toán phân loại email thành **Ham (0 - Thư hợp lệ)** hoặc **Spam (1 - Thư rác)**. Trọng tâm của bài lab là nắm bắt chính xác đường đi từ văn bản thô đến quyết định phân loại, tự cài đặt và hiểu bản chất toán học của các thuật toán phân loại cơ bản (Logistic Regression, Linear SVM, Complement Naive Bayes), tuân thủ nguyên tắc chia tập Train/Test trước khi tiền xử lý dữ liệu để tránh rò rỉ thông tin (Data Leakage), và phân tích các trường hợp dự đoán sai sót trên tập kiểm tra.

> **Ghi chú phương pháp:** Toàn bộ kết luận, bảng biểu và số liệu trong báo cáo này được đối chiếu trực tiếp từ mã nguồn bài lab (`main.py` , các module `src/`) và các tệp kết quả lưu trữ tại `results/metrics/` . Mọi phân tích kỹ thuật bám sát cấu trúc: *Đầu vào → Lý do lựa chọn → Phép biến đổi / Công thức toán → Đầu ra → Giới hạn & Điểm chưa kết luận được.*

Dữ liệu thô: 5.171 bản ghi | Dữ liệu sau lọc sạch: 4.894 bản ghi

Phân tách: Train (80%) / Val (10%) / Test (10%) | Không gian trích xuất: 4.318 chiều → Chọn: 500 chiều

Mô hình tối ưu: Linear SVM (C=1.0, Ngưỡng=0,555) | Test Accuracy: 96,52% | F1-Score: 94,24%

---

1. Đặt vấn đề và tiêu chí đánh giá bài lab

1.1. Phát biểu bài toán

Chương trình tiếp nhận đầu vào là chuỗi ký tự thô đại diện cho nội dung một bức email $x \in \mathcal{X}$ và đưa ra dự đoán nhãn lớp $y \in \{0, 1\}$, trong đó 0 tương ứng với lớp **Ham** (email thông thường, hợp lệ) và 1 tương ứng với lớp **Spam** (thư rác, quảng cáo, lừa đảo). Bên cạnh nhãn dự đoán, mô hình cung cấp một điểm số tin cậy $s \in [0, 1]$ phản ánh mức độ nghi ngờ là thư rác.

1.2. Phân tích mức độ ảnh hưởng của sai lầm

Trong bài toán lọc thư rác thực tế, hai loại sai lầm mang lại hậu quả rất khác nhau:

- **False Positive (FP - Chặn nhầm):** Dự đoán nhầm một email hợp lệ (Ham) thành Spam. Hậu quả là người dùng bị lỡ thông báo tài khoản ngân hàng, trao đổi công việc hoặc thư mời phỏng vấn quan trọng. Đây là loại lỗi cần hạn chế tối đa.
- **False Negative (FN - Bỏ sót):** Để lọt một bức thư rác (Spam) vào hòm thư đến (Inbox). Hậu quả là gây phiền toái hoặc nguy cơ lừa đảo nếu người dùng mở liên kết độc hại.

1.3. Tiêu chuẩn đánh giá và quy tắc quyết định trong bài lab

Để vừa phát hiện thư rác vừa kiểm soát tỷ lệ chặn nhầm, bài lab thiết lập tiêu chí thử nghiệm rõ ràng trên tập Validation:

$$\text{Ràng buộc mục tiêu:} \quad \text{Recall}_{\text{spam}} \geq 0{,}85 \quad (85\%)$$

$$\text{Hàm mục tiêu lựa chọn mô hình:} \quad \max(\text{Accuracy}) \quad \text{sau khi thỏa mãn Recall} \geq 0{,}85$$

Nếu có nhiều cấu hình đạt ngưỡng Recall $\geq 0{,}85$, cấu hình có **Accuracy** cao nhất sẽ được chọn; nếu Accuracy bằng nhau, ưu tiên tiếp theo là **F1-Score**, sau đó là **Precision**, và cuối cùng là giảm thiểu **FPR (False Positive Rate)**. Nếu không có mô hình nào đạt Recall $0{,}85$, mô hình có Recall cao nhất sẽ được chọn trước khi xét đến Accuracy.

> **Vì sao không dùng riêng Accuracy để đánh giá bài toán?**
>
> Trong tập dữ liệu kiểm tra gồm 489 email (343 Ham và 146 Spam), một cách đoán đơn giản là luôn đoán tất cả email là Ham (Majority class baseline) sẽ đạt ngay độ chính xác Accuracy $= 343/489 \approx 70{,}14\%$ mà không cần huấn luyện gì. Tuy nhiên, cách này hoàn toàn vô dụng vì bỏ lọt toàn bộ thư rác (Recall $= 0\%$). Do đó, Accuracy đơn lẻ không phản ánh được chất lượng khi dữ liệu có sự chênh lệch lớp (tỷ lệ spam thực tế ~30%).

2. Thiết kế chương trình và luồng xử lý dữ liệu

Chương trình được thiết kế theo cấu trúc module hóa phân tầng, tách bạch rõ ràng giữa hai luồng xử lý độc lập: **Luồng huấn luyện & thẩm định ngoại tuyến (Offline Training Pipeline)** và **Luồng suy luận phân loại email mới trực tuyến (Online Inference Pipeline)**.

2.1. Luồng huấn luyện và thẩm định (Training & Evaluation Pipeline)

Luồng huấn luyện thực hiện toàn bộ các bước từ tiền xử lý, trích xuất đặc trưng, tối ưu hóa siêu tham số đến đóng gói artifact mô hình tốt nhất:

Dữ liệu CSV thô → Kiểm tra & Vệ sinh → Chia Train / Val / Test → Fit Feature Builder trên Train → Xếp hạng & Chọn 500 đặc trưng → Tuning & Chọn Model trên Val → Kiểm định độc lập trên Test → Đóng gói Bundle (.joblib)

2.2. Luồng suy luận dự đoán email mới (Inference Pipeline)

Khi có một email mới (chuỗi văn bản chưa từng thấy) đưa vào lớp `SpamInferenceService`, chu trình suy luận diễn ra hoàn toàn tự động bằng các tham số đã đóng băng, tuyệt đối không fit lại dữ liệu:

Email thô mới (chuỗi văn bản) → Làm sạch văn bản (SpamTextProcessor) → Transform 4.318 đặc trưng (Builder đã fit) → Lọc đúng 500 cột đặc trưng tối ưu → Linear SVM tính điểm xác suất (Sigmoid) → So sánh với ngưỡng tối ưu 0,555 → Kết luận: Ham / Spam (+ Tín hiệu cảnh báo)

Chi tiết 5 bước kỹ thuật trong luồng suy luận:

1. **Tiếp nhận văn bản:** Nhận chuỗi văn bản email thô ban đầu (bao gồm cả Subject nếu có).
2. **Làm sạch văn bản (Stateless Clean):** Gọi `SpamTextProcessor` áp dụng các regex cố định để chuyển chữ thường, thay token URL/tiền tệ/số, xóa ký tự đặc biệt và lọc stop words, tạo chuỗi `model_text`.
3. **Biến đổi đặc trưng (Stateful Transform):** Gọi `FittedFeatureTransform` áp dụng `EmailFeatureBuilder` đã học từ tập Train (sử dụng từ vựng, vector IDF, thông số min/max scaler đã đóng băng; không fit lại) để tạo ma trận đặc trưng thưa $1 times 4.318$.
4. **Cắt lọc đặc trưng:** Trích xuất đúng 500 cột có chỉ số nằm trong `selected_indices` để tạo ma trận $1 times 500$ khớp chính xác với vector trọng số của mô hình.
5. **Dự đoán và ra quyết định:** Mô hình `Linear SVM` tính điểm qua hàm sigmoid $\sigma(w^T x + b)$; nếu điểm $\geq 0{,}555$ chốt nhãn **SPAM**, ngược lại chốt nhãn **HAM**, đồng thời trả về danh sách các tín hiệu nhạy cảm bề mặt (từ khóa, liên kết, dấu chấm than).

2.3. Bảng phân bổ trách nhiệm của các module trong bài lab

Bảng phân bổ trách nhiệm của các module trong bài lab

Module	Tệp nguồn chính	Trách nhiệm chính	Dữ liệu đầu ra
Data Access	`src/data/loader.py`	Đọc CSV đa bảng mã, chuẩn hóa tên cột, gán mã `source_id` theo chỉ số dòng gốc	DataFrame chuẩn (source_id, label, text, raw_text)
Data Hygiene	`src/data/preprocessing.py`	Kiểm tra tính hợp lệ, khử trùng lặp theo khóa chuẩn hóa, phân tách tập phân tầng	Train, Val, Test splits độc lập
Feature Extraction	`src/features/feature_engineering.py`	TF-IDF unigram/bigram, char n-gram, keywords regex, numeric min-max, lọc Top-K	Ma trận đặc trưng thưa X, đối tượng `EmailFeatureBuilder`
Classifiers	`src/models/classifiers.py`	Tự cài đặt Logistic Regression, Linear SVM, Complement Naive Bayes bằng NumPy/SciPy	Mô hình học máy có phương thức `fit`, `predict_proba`
Tuning & Ensemble	`src/models/tuning.py`, `ensemble.py`	Grid search siêu tham số, tối ưu hóa ngưỡng cắt, Soft Voting Ensemble	Bộ tham số tối ưu, ngưỡng cắt τ, mô hình tổ hợp
Evaluation	`src/evaluation/`	Tính toán chỉ số kiểm định, ma trận nhầm lẫn, biểu đồ phân tích, trích xuất mẫu lỗi	Bảng metrics CSV, biểu đồ PNG trong `results/`
Deployment	`src/deployment/predictor.py`	Nạp bundle `.joblib`, xử lý văn bản email mới, dự đoán và gắn nhãn chẩn đoán	Dictionary kết quả (label, probability, signals, suspected_cause)

Bảo vệ nhãn dữ liệu: Các cột thông tin nhãn như `label`, `label_num` và mã định danh `source_id` bị loại bỏ hoàn toàn khỏi ma trận đặc trưng X. Điều này ngăn chặn việc mô hình học dựa trên thông tin đáp án.

3. Nạp, kiểm tra dữ liệu và phân tách tập độc lập

3.1. Nguồn dữ liệu thô và quá trình nạp

Dữ liệu thực nghiệm sử dụng tệp `data/raw/spam_ham_dataset.csv`, có nguồn gốc từ tập *Spam Mails Dataset* (Kaggle venky73, trích xuất từ tập Enron-Spam). Văn bản email bao gồm cả phần tiêu đề `Subject:` và nội dung thư được nối liền nhau thành một trường chuỗi ký tự duy nhất.

Quy trình thực thi trong `main.py` gọi trực tiếp hàm `load_raw_data()` rồi chuyển sang `clean_email_split()`. Loader hỗ trợ giải mã UTF-8 (có hoặc không có BOM) và fallback sang Latin-1 khi gặp lỗi mã hóa ký tự byte đặc thù.

3.2. Vệ sinh dữ liệu thô (Data Hygiene)

Các tiêu chí kiểm tra dữ liệu được thiết kế không chồng lấn nhằm bảo đảm tính toàn vẹn trước khi phân chia tập:

Kết quả kiểm tra và vệ sinh dữ liệu (Đối chiếu trực tiếp `data_audit.json`)

Hạng mục kiểm tra	Số lượng dòng	Tỷ lệ	Ghi chú kỹ thuật
Tổng bản ghi ban đầu trong file CSV	5.171	100,00%	Nguồn data/raw/spam_ham_dataset.csv
Bản ghi nhãn không hợp lệ (ngoài ham/spam)	0	0,00%	Không có dữ liệu lỗi nhãn
Bản ghi văn bản rỗng sau khi cắt khoảng trắng	0	0,00%	Không có email rỗng
Nhóm xung đột nhãn (cùng văn bản, khác nhãn)	0	0,00%	Không phát hiện mâu thuẫn nhãn
Bản ghi trùng lặp nội dung cùng nhãn bị loại	277	5,36%	Loại bỏ theo khóa Unicode NFKC
Số bản ghi sạch giữ lại sau kiểm tra	**4.894**	**94,64%**	Dữ liệu đưa vào phân tách tập

Khóa khử trùng lặp được xây dựng bằng cách áp dụng chuẩn hóa **Unicode NFKC**, gộp các khoảng trắng liền kề, giữ nguyên hoa/thường và dấu câu. Các email có nội dung giống hệt nhau về khóa này chỉ giữ lại bản ghi xuất hiện đầu tiên. Thao tác này ngăn chặn hiện tượng cùng một email bị phân bổ vào cả tập Train và Test.

> **Giới hạn của bước khử trùng lặp:** Khử trùng lặp theo khóa chính xác **không chứng minh các mẫu dữ liệu là độc lập đồng nhất (i.i.d)**. Phương pháp này không phát hiện được các email gần trùng (near-duplicates) hoặc các email thuộc cùng một chiến dịch thư rác chỉ khác biệt một vài ký tự ngẫu nhiên.

3.3. Phân tách tập phân tầng (Stratified Splitting 80/10/10)

Sau khi vệ sinh, 4.894 bản ghi được phân tách ngẫu nhiên có phân tầng (Stratified) thành ba tập độc lập bằng hàm `stratified_split_dataframe()` với `random_state=42`:

Phân bổ mẫu và tỷ lệ phân lớp giữa các tập dữ liệu

Tập dữ liệu	Tỷ lệ cấu hình	Tổng số dòng	Số lượng Ham (0)	Số lượng Spam (1)	Vai trò trong bài thực hành

Tập dữ liệu	Tỷ lệ cấu hình	Tổng số dòng	Số lượng Ham (0)	Số lượng Spam (1)	Vai trò trong bài thực hành
Tập huấn luyện (Train)	80%	3.916	2.746 (70,12%)	1.170 (29,88%)	Fit bộ trích xuất đặc trưng & huấn luyện mô hình
Tập thẩm định (Validation)	10%	489	343 (70,14%)	146 (29,86%)	Chọn K đặc trưng, tinh chỉnh siêu tham số & tối ưu ngưỡng
Tập kiểm tra (Test)	10%	489	343 (70,14%)	146 (29,86%)	Kiểm tra khả năng tổng quát hóa độc lập
Tổng cộng	**100%**	**4.894**	**3.432 (70,13%)**	**1.462 (29,87%)**	Phân tầng giữ nguyên tỷ lệ nhãn qua seed 42

Việc phân tầng đảm bảo tỷ lệ spam trong cả ba tập giữ ổn định ở mức ~29,87%, loại trừ rủi ro phân phối nhãn bị lệch giữa các tập thử nghiệm.

4. Tiền xử lý văn bản (SpamTextProcessor)

4.1. Bản chất: Bộ xử lý không trạng thái (Stateless Processor)

Lớp `SpamTextProcessor` trong `src/data/preprocessing.py` được thiết kế hoàn toàn **không lưu trạng thái (không có hàm `fit`)**. Mọi thao tác xử lý văn bản đều dựa trên các biểu thức chính quy (regex) cố định và từ điển stop words có sẵn. Lớp này xử lý từng văn bản độc lập mà không tính toán bất kỳ giá trị thống kê tích lũy nào trên toàn bộ tập dữ liệu.

> **Chuẩn hóa văn bản trước khi chia tập có chắc chắn gây rò rỉ dữ liệu (Data Leakage) không?**
>
> Về mặt toán học, việc làm sạch cục bộ từng văn bản bằng các luật regex và stop words cố định (ví dụ: chuyển chữ thường, thay link bằng `urltoken`) **không gây rò rỉ thông tin phân phối giữa các mẫu**. Tuy nhiên, theo nguyên tắc chuẩn của bài lab, việc chia tập được thực hiện trước rồi mới tiền xử lý nhằm bảo đảm ranh giới dữ liệu rõ ràng và loại trừ mọi nguy cơ rò rỉ thông tin ở các bước trích xuất đặc trưng thống kê tiếp theo.

4.2. Các bước chuẩn hóa chuỗi ký tự

1. **Chuyển chữ thường (Lowercasing):** Đưa toàn bộ ký tự về chữ thường để đồng nhất từ vựng.
2. **Thay thế liên kết web (URL Tokenization):** Chuyển các đường dẫn mạng khớp mẫu `https?://\S+|www\.\S+` thành token `urltoken`.
3. **Thay thế tiền tệ và số tiền (Money Tokenization):** Chuyển các biểu thức chứa ký hiệu tiền tệ `[\$£€¥]` hoặc từ khóa tài chính kèm con số thành token `moneytoken`.
4. **Thay thế con số đơn lẻ (Number Tokenization):** Chuyển các chuỗi số nguyên hoặc số thập phân còn lại thành token `numbertoken`.
5. **Lọc ký tự đặc biệt & chuẩn hóa khoảng trắng:** Loại bỏ các dấu câu không cần thiết, chuyển chuỗi về các từ đơn cách nhau bằng một dấu cách.
6. **Lọc bỏ Stop Words:** Loại bỏ các từ dừng tiếng Anh phổ biến (danh sách chuẩn gồm 128 từ như *the, is, at, which, on*...) không mang ý nghĩa phân loại thư rác.

4.3. Ví dụ biến đổi văn bản thực tế

Văn bản email thô ban đầu:

`"URGENT! Claim $100 at https://example.com now!"`

Văn bản sau chuẩn hóa (`model_text`):

`"urgent claim moneytoken urltoken now"`

Lưu ý về chuẩn hóa: Khối ký tự n-gram phía sau nhận chuỗi `model_text` đã được chuẩn hóa chứ không nhận chuỗi thô. Do đó, các cách viết biến tướng như `"w!n fr3e"` sau khi qua bộ chuẩn hóa sẽ trở thành `"w n fr3e"` . Chuỗi ký tự n-gram trích xuất từ `model_text` không thể khôi phục lại cấu trúc ký tự đặc biệt ban đầu của người gửi.

5. Biểu diễn đặc trưng số học (Feature Engineering)

5.1. Bản chất: Bộ trích xuất có trạng thái (Stateful Feature Builder)

Khác với bộ tiền xử lý văn bản, lớp `EmailFeatureBuilder` trong `src/features/feature_engineering.py` là một thành phần **có trạng thái (stateful)**. Đối tượng này bắt buộc phải gọi `fit()` **chỉ trên tập Train** để học từ vựng, tính vector tần suất nghịch đảo tài liệu IDF, học ngưỡng giới hạn độ dài văn bản và các giá trị cực trị Min/Max của các biến số học. Khi biến đổi tập Validation, Test hoặc email mới, builder chỉ áp dụng phương thức `transform()` dựa trên các tham số đã đóng băng.

5.2. Cấu trúc không gian đặc trưng ban đầu (4.318 chiều)

Không gian đặc trưng đầu vào được ghép từ 5 khối độc lập:

1. TF-IDF Unigram (Tối đa 3.000 đặc trưng)

Đặc trưng tần số từ đơn được tính theo công thức TF log-count kết hợp chuẩn hóa IDF:

$$\mathrm{tf}(t,d) = \ln(1 + \mathrm{count}(t,d))$$

$$\mathrm{idf}(t) = \ln\left(\frac{1+N}{1+\mathrm{df}(t)}\right) + 1$$

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t)$$

Trong đó: $N = 3.916$ là **tổng số tài liệu trong tập Train**; $\mathrm{df}(t)$ là số tài liệu trong tập Train chứa từ t; ngưỡng cắt tần suất tối thiểu $\min_\mathrm{df} = 2$. Giá trị 3.000 là trần số lượng từ vựng unigram có tần số cao nhất (`max_unigram_features`), không phải là giá trị N. Mỗi vector văn bản sau đó được chuẩn hóa theo chuẩn L_2.

2. TF-IDF Bigram (Tối đa 800 đặc trưng)

Trích xuất các cặp 2 từ liền kề có tần suất xuất hiện $\mathrm{df} \geq 2$ trong tập Train, giúp mô hình ghi nhận các cụm từ đặc thù như "*credit card*", "*act now*", "*attached file*".

3. Character n-gram (Tối đa 500 đặc trưng, phạm vi n từ 3 đến 5)

Trích xuất trên chuỗi `model_text` sau chuẩn hóa, giúp nắm bắt các cụm gốc từ hoặc hình thái từ vựng ngay cả khi từ đó bị viết sai chính tả nhẹ.

4. Keyword Features (10 đặc trưng nhị phân)

Gồm đúng **10 biến nhị phân** $\{0{,}0;\, 1{,}0\}$ kiểm tra sự xuất hiện của các nhóm mẫu biểu thức chính quy (không đếm số lần và không phải chỉ là 10 từ đơn):

- `kw_free` : `\bfree\b`
- `kw_win` : `\b(?:win|winner|won|winning)\b`
- `kw_prize` : `\b(?:prize|reward|gift|bonus)\b`
- `kw_claim` : `\b(?:claim|redeem|collect)\b`
- `kw_urgent` : `\b(?:urgent|immediately|act now|important)\b`
- `kw_call` : `\b(?:call|txt|phone|ring)\b`
- `kw_cash` : `\b(?:cash|money|dollar|\$|pound|credit)\b`
- `kw_guarantee` : `\b(?:guarantee|guaranteed)\b`
- `kw_offer` : `\b(?:offer|discount|promo|deal)\b`
- `kw_click` : `\b(?:click|link|visit|website)\b`

5. Numeric & Structural Features (8 đặc trưng cấu trúc)

Trích xuất các tín hiệu thống kê bề mặt từ chuỗi văn bản:

- 2 biến nhị phân: `has_url` , `has_number` .
- 5 biến đếm số lượng: `digit_count` , `special_char_count` , `exclamation_count` , `dollar_count` , `model_length` . Các biến này được biến đổi phi tuyến qua hàm $\ln(1+x)$ rồi chuẩn hóa Min-Max scaling về đoạn $[0, 1]$ theo giá trị cực trị học từ tập Train.
- 1 biến tỷ lệ: `uppercase_ratio` (tỷ lệ ký tự viết hoa trong văn bản gốc) được Min-Max scaling về $[0, 1]$.

5.3. Giới hạn độ dài văn bản (Length Capping)

Để tránh trường hợp một số email quá dài làm lệch phân phối đặc trưng, chương trình áp dụng cắt độ dài tại **phân vị 99% (`cap_quantile=0.99`)** của tập Train. Giá trị ngưỡng tính được là **791 từ của chuỗi `model_text` sau chuẩn hóa** (không phải số từ của văn bản thô). Mọi văn bản dài hơn 791 từ chuẩn hóa đều bị cắt ngắn ở từ thứ 791 trước khi trích xuất vector.

6. Lựa chọn đặc trưng và tiêu chuẩn cân bằng (Top-K Selection)

6.1. Phương pháp tính điểm quan trọng kết hợp (Combined Importance)

Trong quy trình thực thi tại `main.py`, hàm `compute_combined_importance(X_train_full, y_train)` được gọi mà không truyền mô hình Naive Bayes phụ. Do đó, điểm số quan trọng của từng đặc trưng j được tính theo công thức fallback kết hợp giữa kiểm định Chi-bình phương (χ^2) và tổng trọng số cột:

$$\text{Score}_j = 0{,}90 \times \text{Norm}(\chi^2_j) + 0{,}10 \times \text{Norm}\left(\ln(1 + \text{column_sum}_j)\right)$$

Trong đó:

- χ^2_j: Giá trị thống kê Chi-bình phương đo lường mức độ phụ thuộc giữa đặc trưng j và nhãn phân loại trên tập Train.
- column_sum_j: Tổng giá trị trọng số của đặc trưng j trên toàn bộ các mẫu của tập Train ($\sum_{i=1}^{N} X_{i,j}$). Đây là tổng cường độ đặc trưng, không phải số lượng tài liệu chứa từ.
- $\text{Norm}(\cdot)$: Phép chuẩn hóa vector về đoạn $[0, 1]$ bằng cách chia cho giá trị lớn nhất trong không gian đặc trưng.

6.2. Tiêu chí lựa chọn K đặc trưng tối ưu

Sau khi sắp xếp 4.318 đặc trưng theo thứ tự điểm giảm dần, chương trình thử nghiệm các ngưỡng kích thước $K \in [500, 1000, 1500, 2000, 2500, 3000, 4318]$. Mỗi cấu hình được đánh giá nhanh bằng mô hình Complement Naive Bayes ($\alpha = 1{,}0$, ngưỡng cố định $\tau = 0{,}5$) trên tập Validation.

Quy tắc lựa chọn ưu tiên đạt ràng buộc $\text{Recall}_{\text{val}} \geq 0{,}85$. Khi ràng buộc được thỏa mãn, cấu hình được xếp hạng theo **Hàm điểm cân bằng (Balance Score)**:

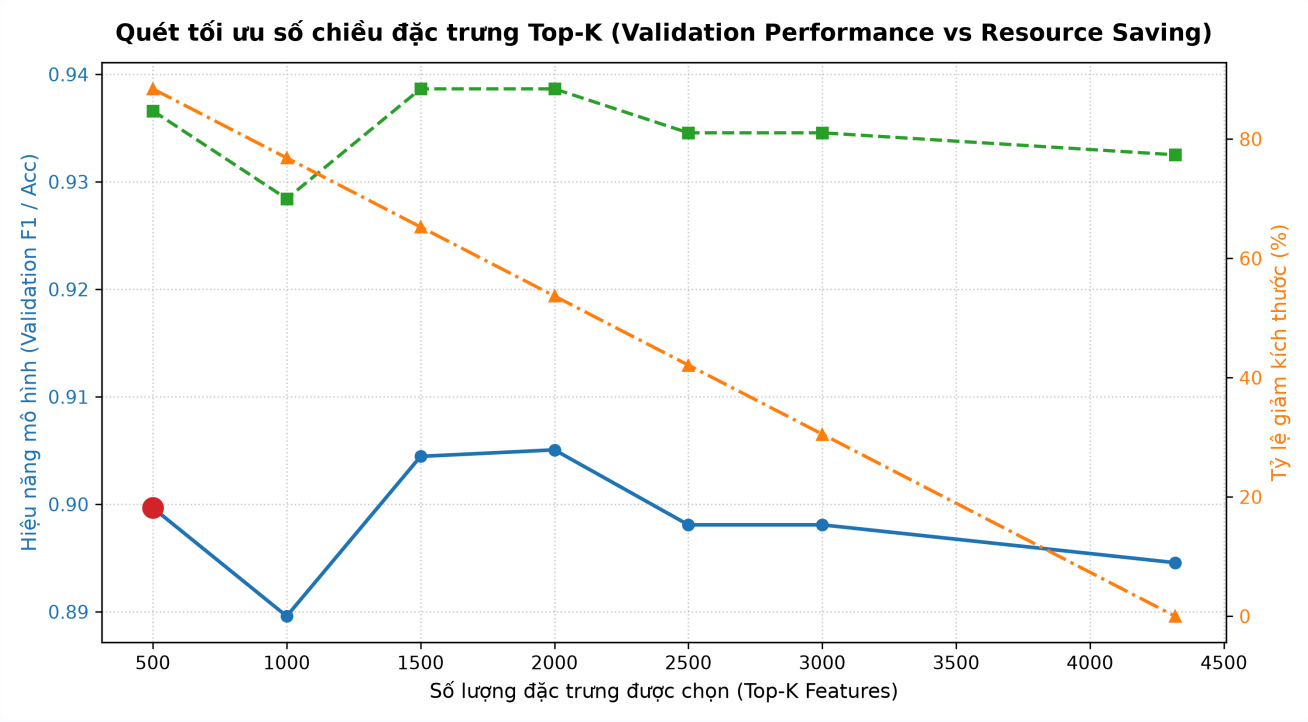
$$\text{Balance Score} = 0{,}45 \times \text{F1} + 0{,}35 \times \text{Accuracy} + 0{,}20 \times \text{Saving Fraction} - 0{,}50 \times \text{Positive Gap}$$

$$\text{với} \quad \text{Saving Fraction} = \frac{4318 - K}{4318}, \quad \text{Positive Gap} = \max\left(0,\ \text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{val}}\right)$$

Kết quả thực nghiệm quét ngưỡng số lượng đặc trưng Top-K (Trích xuất từ `top_k_sweep.csv`)

Số đặc trưng (K)	Tỷ lệ giảm chiều	Val Accuracy	Val Precision	Val Recall	Val F1-Score	Train-Val Gap	Đạt Target	Balance Score
500	88,42%	93,66%	85,28%	95,21%	89,97%	0,0000	True	**0,9095** (Được chọn)
1.000	76,84%	92,84%	82,46%	96,58%	88,96%	0,0075	True	0,8752
1.500	65,26%	93,87%	84,52%	97,26%	90,45%	0,0000	True	0,8661
2.000	53,68%	93,87%	84,12%	97,95%	90,51%	0,0013	True	0,8425

Số đặc trưng (K)	Tỷ lệ giảm chiều	Val Accuracy	Val Precision	Val Recall	Val F1-Score	Train-Val Gap	Đạt Target	Balance Score
2.500	42,10%	93,46%	83,93%	96,58%	89,81%	0,0075	True	0,8117
3.000	30,52%	93,46%	83,93%	96,58%	89,81%	0,0093	True	0,7877
4.318 (Toàn bộ)	0,00%	93,25%	83,83%	95,89%	89,46%	0,0085	True	0,7247



Hình 1: Đánh giá sự biến thiên của các chỉ số hiệu năng và điểm cân bằng theo số lượng đặc trưng K

Lý do lựa chọn: Cấu hình $K = 500$ được lựa chọn vì đạt điểm cân bằng cao nhất ($0{,}9095$). Nhờ giảm $88{,}42\%$ số chiều dữ liệu, mô hình loại bỏ bớt từ hiếm gây nhiễu, giúp việc huấn luyện nhanh hơn và giữ Train-Val Gap bằng $0{,}0000$. Dù cấu hình $K = 1.500$ và $K = 2.000$ có F1-Score trên Validation cao hơn ($90{,}45\%$ và $90{,}51\%$), nhưng điểm cân bằng tổng thể thấp hơn ($0{,}8661$ và $0{,}8425$) do số chiều dữ liệu còn lớn hơn đáng kể.

7. Nền tảng toán học của ba thuật toán tự cài đặt

Toàn bộ 3 thuật toán phân loại được cài đặt trực tiếp bằng mã nguồn Python kết hợp NumPy/SciPy trong `src/models/classifiers.py`, không dùng các class mô hình có sẵn của scikit-learn để học rõ bản chất tính toán.

7.1. Cân bằng trọng số lớp (Class Weights)

Để xử lý việc lệch tỷ lệ giữa hai lớp, các thuật toán phân loại tuyến tính dùng trọng số nghịch đảo tần suất lớp:

$$a_c = \frac{N}{2 \times N_c}$$

Trên tập Train ($N = 3.916$) : $a_{\text{spam}} = \frac{3916}{2 \times 1170} \approx 1{,}6735$; $a_{\text{ham}} = \frac{3916}{2 \times 2746} \approx 0{,}7130$

7.2. Logistic Regression (LR)

Mô hình tính xác suất hậu nghiệm thông qua hàm sigmoid:

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{với } z = w^T x + b$$

Hàm mất mát Binary Cross-Entropy có trọng số lớp và điều quy L_2 :

$$J(w, b) = -\frac{1}{N} \sum_{i=1}^{N} a_{y_i} \left[y_i \ln \sigma(z_i) + (1 - y_i) \ln(1 - \sigma(z_i)) \right] + \frac{\lambda}{2} \|w\|_2^2, \quad \text{với } \lambda = \frac{1}{C \cdot N}$$

Gradient cập nhật: $\nabla_w J = \frac{1}{N} \sum_{i=1}^{N} a_{y_i} \left(\sigma(z_i) - y_i \right) x_i + \lambda w$; $\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} a_{y_i} \left(\sigma(z_i) - y_i \right)$

7.3. Linear Support Vector Machine (Linear SVM)

Mô hình tìm khoảng cách biên lề tối ưu với hàm mất mát Squared Hinge loss:

$$\min_{w,b} \mathcal{L}(w, b) = \frac{\lambda}{2} \|w\|_2^2 + \frac{1}{N} \sum_{i=1}^{N} a_{\tilde{y}_i} \max\left(0,\, 1 - \tilde{y}_i (w^T x_i + b)\right)^2, \quad \tilde{y}_i \in \{-1, +1\}, \quad \lambda = \frac{1}{C \cdot N}$$

Gradient thành phần lỗi: nếu $1 - \tilde{y}_i z_i > 0 \implies \nabla_w \ell_i = -2 a_{\tilde{y}_i} (1 - \tilde{y}_i z_i) \tilde{y}_i x_i$

Chuyển đổi điểm số sang xác suất: Khoảng cách biên lề $z = w^T x + b$ được chuyển sang thang đo $[0, 1]$ qua hàm sigmoid $s = \sigma(z) = \frac{1}{1+e^{-z}}$. Ngưỡng quyết định tối ưu $\tau = 0{,}555$ được áp dụng trên giá trị sigmoid này, tương đương với khoảng cách biên lề: $z = \ln\left(\frac{0{,}555}{1-0{,}555}\right) \approx +0{,}221$.

7.4. Complement Naive Bayes (CNB)

CNB được thiết kế để cải thiện Naive Bayes khi dữ liệu mất cân bằng. Thay vì ước lượng xác suất từ xuất hiện trong chính lớp đó, CNB ước lượng tham số từ các tài liệu **không thuộc lớp** c (ký hiệu là $\bar{c}$):

$$\theta_{\bar{c},j} = \frac{\alpha + \sum_{i \in \bar{c}} x_{i,j}}{\alpha \cdot D + \sum_{j'=1}^{D} \sum_{i \in \bar{c}} x_{i,j'}}$$

$$\text{Điểm số log-odds trong mã nguồn thực tế:} \quad z_c = \ln P(c) - \sum_{j=1}^{D} x_j \ln \theta_{\bar{c},j}$$

Mã nguồn dự án tính điểm số trực tiếp từ công thức trên rồi chuyển sang xác suất bằng hàm softmax 2 lớp (không áp dụng bước chuẩn hóa chia tổng độ dài vector tham số).

7.5. Thuật toán tối ưu hóa Gradient Descent

Đối với LR và SVM, bộ tối ưu hóa áp dụng kỹ thuật xấp xỉ bậc hai Barzilai-Borwein (bước phổ) kết hợp kiểm tra suy giảm Armijo (Backtracking Line Search) để tự động chỉnh độ dài bước nhảy η_t. Quá trình lặp dừng lại khi đạt số vòng tối đa `max_iter` hoặc khi chuẩn gradient đạt ngưỡng dung sai $\|\nabla J\|_2 \leq \text{tol}$ (với $\text{tol} = 10^{-4}$).

8. Tinh chỉnh siêu tham số, mô hình kết hợp và tối ưu ngưỡng

8.1. Phạm vi lưới tinh chỉnh siêu tham số (Grid Search)

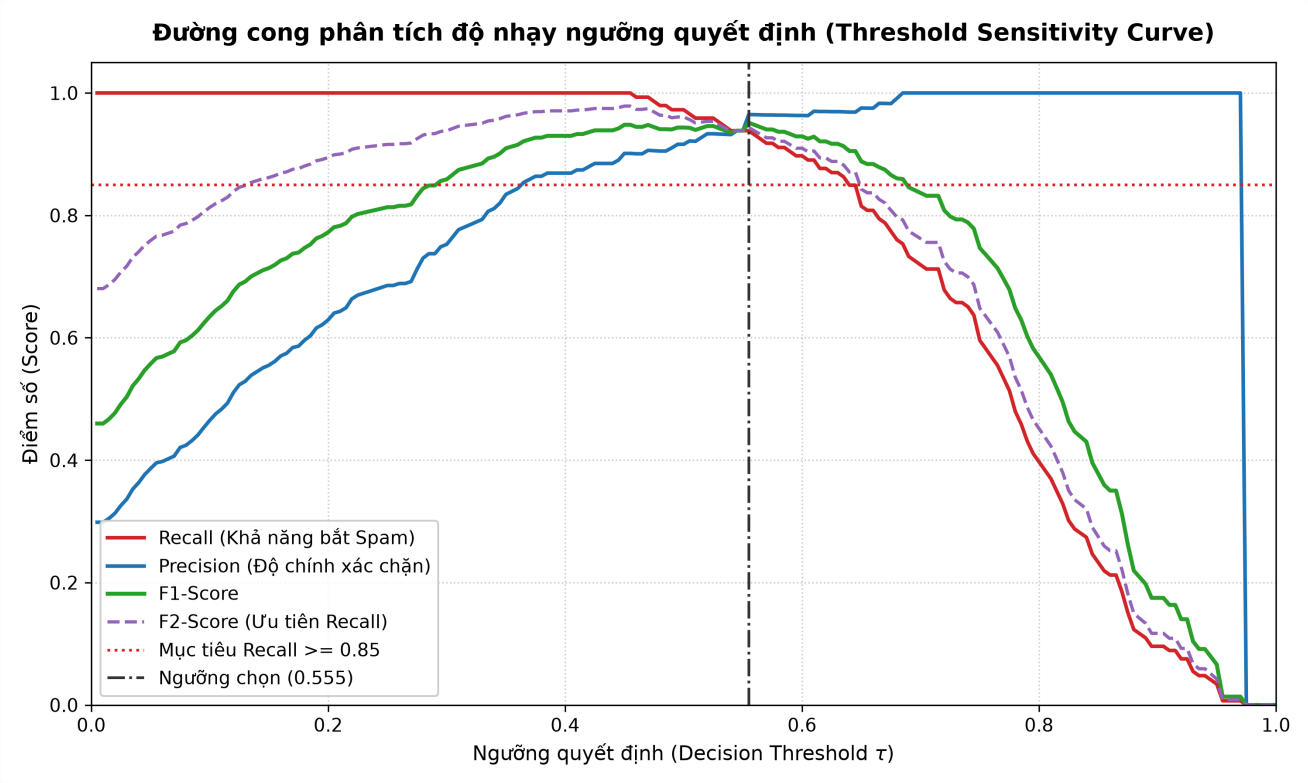
Không gian tìm kiếm tham số được cấu hình trong `src/models/tuning.py` :

- **Logistic Regression:** Thử $C \in \{0{,}5; 1{,}0; 2{,}0\}$. Tham số $\text{lr} = 0{,}1$ và $\max_\text{iter} = 200$ được giữ cố định.
- **Linear SVM:** Thử $C \in \{0{,}5; 1{,}0; 2{,}0\}$. Tham số $\text{lr} = 0{,}05$, $\max_\text{iter} = 150$, loss= `squared_hinge` được giữ cố định.
- **Complement Naive Bayes:** Thử tham số làm trơn Laplace $\alpha \in \{0{,}5; 1{,}0; 2{,}0\}$.

Lưu ý kỹ thuật: Lưới tìm kiếm thực tế chỉ thay đổi tham số C hoặc α; tốc độ học ban đầu và số vòng lặp tối đa không được dò nhiều mức.

8.2. Quy trình quét tối ưu hóa ngưỡng cắt (Threshold Tuning)

Với mỗi mô hình, chương trình quét qua lưới gồm **200 giá trị ngưỡng cách đều tuyến tính** từ $0{,}005$ đến $1{,}000$ (bước nhảy $\Delta\tau = 0{,}005$), không phải quét theo phân vị điểm số. Tại mỗi ngưỡng τ, các chỉ số trên tập Validation được tính toán. Ngưỡng tối ưu được chọn theo quy tắc: đạt $\text{Recall}_{\text{val}} \geq 0{,}85$, sau đó tối đa hóa Accuracy, F1, Precision và tối thiểu hóa FPR.



Hình 2: Phân tích độ nhạy của Precision, Recall và F1-Score theo ngưỡng cắt xác suất trên tập Validation

8.3. Mô hình kết hợp (SpamVotingEnsemble)

Lớp `SpamVotingEnsemble` kết hợp mềm (Soft Voting) điểm xác suất từ 3 mô hình cơ sở:

- **Ensemble Baseline:** Trọng số kết hợp $w_{\text{LR}} = 1{,}0$, $w_{\text{SVM}} = 1{,}0$, $w_{\text{CNB}} = 1{,}2$. Ngưỡng tối ưu trên Validation là $\tau = 0{,}605$.
- **Ensemble Tuned:** Trọng số kết hợp đồng đều $1{,}0 : 1{,}0 : 1{,}0$. Ngưỡng tối ưu trên Validation là $\tau = 0{,}605$.

Ghi chú: Các hàm hỗ trợ mô hình cây như Random Forest và Gradient Boosting có sẵn trong mã nguồn tham khảo nhưng không đưa vào pipeline thực thi chính của bài lab.

9. Vòng đời huấn luyện và giám sát thực nghiệm

Lần chạy thực nghiệm hoàn chỉnh của pipeline được ghi nhận tại tệp `results/metrics/run_summary.json`:

- **Thời điểm thực thi:** `2026-11-05T03:51:50.253795+00:00` (Tương ứng 10:51:50 ngày 05/11/2026 giờ Việt Nam).
- **Môi trường thực thi:** Python 3.12.10 trên Windows; các thư viện sử dụng: NumPy 2.5.3, SciPy 1.18.1, Pandas 3.0.6, Joblib 1.6.0, Matplotlib 3.11.2, Seaborn 0.13.2, Scikit-learn 1.9.1.
- **Mã băm SHA-256 dữ liệu nạp:** `471d86ef23d07af0f37e350a754df1e4c700fd4804aebe3a5ad6b751c237455d`.
- **Khớp mã nguồn:** Toàn bộ mã băm SHA-256 của 22 tệp mã nguồn ghi nhận trong artifact khớp chính xác với mã nguồn hiện tại của bài lab.

10. Kết quả thực nghiệm trên tập Validation và Test

10.1. Đánh giá lựa chọn mô hình trên tập Validation

Bảng so sánh 8 cấu hình ứng viên dựa trên kết quả chạy trên tập Validation (489 mẫu):

Kết quả thẩm định và xếp hạng mô hình trên tập Validation (Trích xuất từ `validation_model_selection.csv`)

Hạng	Mô hình ứng viên	Cấu hình tham số	Ngưỡng tối ưu	Đạt Target	Val Accuracy	Val Precision	Val Recall	Val F1-Score	Val F2-Score	Train-Val Gap
1	**Linear SVM (Default)**	**Mặc định**	**0,555**	**True**	**97,14%**	**96,48%**	**93,84%**	**95,14%**	**94,35%**	**0,0000**
2	Linear SVM (Tuned)	Grid Search (C=1.0)	0,555	True	97,14%	96,48%	93,84%	95,14%	94,35%	0,0000
3	SpamVotingEnsemble (Tuned)	Soft Voting (Trọng số đều)	0,605	True	96,73%	94,52%	94,52%	94,52%	94,52%	0,0000
4	Logistic Regression (Tuned)	Grid Search (C=2.0)	0,650	True	96,73%	95,14%	93,84%	94,48%	94,09%	0,0000
5	SpamVotingEnsemble	Soft Voting (Trọng số 1:1:1.2)	0,605	True	96,32%	93,24%	94,52%	93,88%	94,26%	0,0000
6	Logistic Regression	Mặc định (C=1.0)	0,590	True	96,11%	92,05%	95,21%	93,60%	94,56%	0,0000
7	Complement Naive Bayes (Tuned)	Grid Search (alpha=0.5)	0,755	True	94,89%	90,60%	92,47%	91,53%	92,09%	0,0000
8	Complement Naive Bayes	Mặc định (alpha=1.0)	0,770	True	94,68%	90,54%	91,78%	91,16%	91,53%	0,0000

Lý do chọn Linear SVM: Cả Linear SVM Baseline và Linear SVM Tuned đều đạt kết quả tốt nhất (Accuracy $97{,}14\%$, Recall $93{,}84\%$, F1 $95{,}14\%$) với cùng $C = 1{,}0$ và cùng ngưỡng $\tau = 0{,}555$. Theo thứ tự sắp xếp ổn định, mô hình cơ sở đứng trước được chốt làm mô hình cuối cùng.

10.2. Đánh giá kiểm tra độc lập trên tập Test

Sau khi chốt mô hình và ngưỡng $\tau = 0{,}555$ từ tập Validation, tập Test (489 mẫu độc lập) được đưa vào kiểm tra:

So sánh kết quả kiểm tra trên tập Test giữa các mô hình (Trích xuất từ `test_models_comparison.csv`)

Mô hình	Accuracy	Precision	Recall	F1-Score	F-Beta (F2)	ROC-AUC	Được chọn từ Validation?

Mô hình	Accuracy	Precision	Recall	F1-Score	F-Beta (F2)	ROC-AUC	Được chọn từ Validation?
Linear SVM (Default)	**96,52%**	**93,29%**	**95,21%**	**94,24%**	**94,82%**	**99,46%**	**Winner (Chốt từ Validation)**
Linear SVM (Tuned)	96,52%	93,29%	95,21%	94,24%	94,82%	99,46%	Đồng hạng (C=1.0)
Logistic Regression (Tuned)	96,73%	92,21%	97,26%	94,67%	96,21%	99,31%	Không chọn vì điểm Validation thấp hơn
SpamVotingEnsemble (Tuned)	95,91%	90,38%	96,58%	93,38%	95,27%	99,31%	Ensemble kết hợp
SpamVotingEnsemble	95,30%	89,17%	95,89%	92,41%	94,47%	99,21%	Ensemble mặc định
Logistic Regression	94,48%	86,06%	97,26%	91,32%	94,79%	99,09%	Baseline LR
Complement Naive Bayes (Tuned)	93,87%	88,67%	91,10%	89,86%	90,60%	98,04%	Tuned CNB
Complement Naive Bayes	93,66%	88,59%	90,41%	89,49%	90,04%	97,89%	Baseline CNB

> **Vì sao mô hình Tuned Logistic Regression có kết quả Test cao hơn (Accuracy 96,73%, F1 94,67%) nhưng không được chọn?**
>
> Theo nguyên tắc đánh giá khách quan của học máy, việc lựa chọn mô hình và tối ưu hóa siêu tham số phải được chốt **dựa trên tập Validation**. Trên tập Validation, Linear SVM có Accuracy ($97,14\%$) và F1 ($95,14\%$) cao hơn Tuned LR ($96,73\%$ và $94,48\%$). Việc đổi sang chọn Tuned LR sau khi đã xem kết quả Test sẽ làm sai lệch tính độc lập của tập kiểm tra (hiện tượng Data Snooping).

10.3. Chi tiết ma trận nhầm lẫn của mô hình được chọn (Linear SVM)

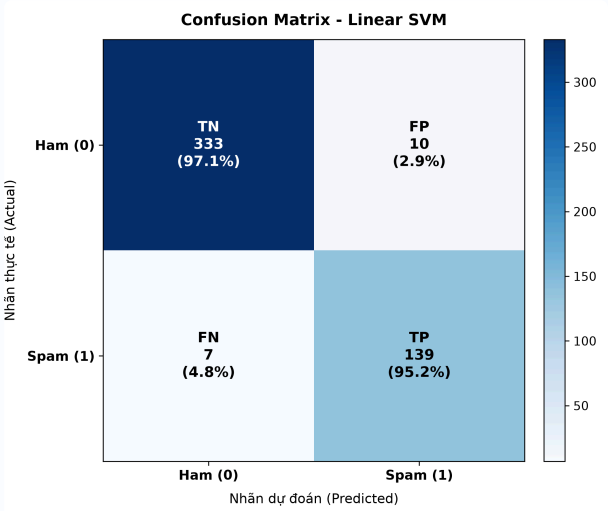
Ma trận nhầm lẫn của Linear SVM trên tập Test (Tổng số mẫu = 489)

Thực tế (Ground Truth)	Dự đoán của mô hình (Predicted)		Tổng thực tế
	Dự đoán Ham (0)	Dự đoán Spam (1)	
Thực tế là Ham (0)	**TN = 333 (68,10%)**	**FP = 10 (2,05%)**	343 (70,14%)
Thực tế là Spam (1)	**FN = 7 (1,43%)**	**TP = 139 (28,43%)**	146 (29,86%)
Tổng dự đoán	340 (69,53%)	149 (30,47%)	489 (100,00%)

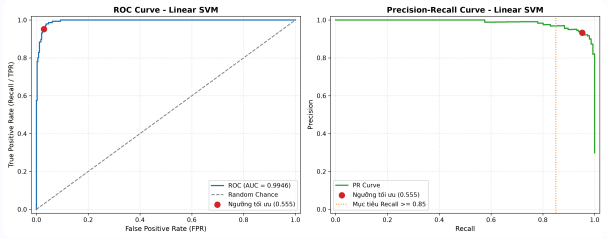
Tỷ lệ chính xác (Accuracy): 96,52% | Độ chuẩn xác (Precision): 93,29% | Độ bao phủ (Recall): 95,21%

Điểm F1-Score: 94,24% | Điểm F2-Score (Ưu tiên Recall): 94,82% | Độ đặc hiệu (Specificity): 97,08%

Tỷ lệ dương tính giả (FPR): 2,92% | Chỉ số diện tích ROC-AUC: 99,46%



Hình 3: Ma trận nhầm lẫn của Linear SVM trên tập Test



Hình 4: Đường cong ROC và Precision-Recall trên tập Test

11. Phân tích chi tiết các mẫu dự đoán sai thực tế (Error Analysis)

Trên tập Test gồm 489 mẫu, mô hình Linear SVM có đúng **17 mẫu phân loại sai** (tỷ lệ lỗi $3{,}48\%$), bao gồm 10 lỗi False Positive và 7 lỗi False Negative. Dưới đây là phân tích chi tiết trích xuất trực tiếp từ tệp `results/metrics/misclassified_samples.csv`:

11.1. Nhóm False Positive (Chặn nhầm 10 email hợp lệ thành Spam)

Danh sách 10 trường hợp False Positive trên tập Test

Source ID	Độ dài (từ thô)	Điểm xác suất (Prob)	Dấu hiệu văn bản	Phân tích nguyên nhân
1823	161	0,6706	Dấu chấm than (3 lần)	Email công việc nhưng dùng nhiều câu cảm thán và từ ngữ giao dịch dồn dập
1940	49	0,6300	Dấu chấm than (2 lần)	Email thông báo ngắn nội bộ, cấu trúc câu ngắn kèm dấu chấm than kích hoạt điểm phạt
3816	76	0,5664	Từ vựng trùng lặp Spam	Chứa các thuật ngữ kỹ thuật trùng với từ khóa xuất hiện nhiều trong thư rác
4747	12	0,6007	Đoạn văn ngắn, thiếu ngữ cảnh	Email quá ngắn (12 từ), một vài token đơn lẻ kích hoạt trọng số dương vượt ngưỡng
2504	143	0,6576	Từ vựng trùng lặp Spam	Trao đổi về số liệu và kế hoạch tài chính, trùng các đặc trưng số học
3841	16	0,6810	Đoạn văn ngắn, thiếu ngữ cảnh	Email trao đổi ngắn (16 từ), thiếu các từ vựng ham để kéo điểm xuống
2375	291	0,7516	Từ vựng trùng lặp Spam	Thảo luận dự án dài nhưng xuất hiện nhiều thuật ngữ tiếp thị
2765	101	0,6294	Từ vựng trùng lặp Spam	Trao đổi nội bộ chứa các đường link hoặc từ khóa tương tự thông báo tự động

Source ID	Độ dài (từ thô)	Điểm xác suất (Prob)	Dấu hiệu văn bản	Phân tích nguyên nhân
4652	65	0,5980	Từ vựng trùng lặp Spam	Nội dung xác nhận cuộc họp nhưng dùng từ ngữ tương tự mẫu chào hàng
278	850	0,6261	Từ khóa nhạy cảm (free, call)	Email kỹ thuật rất dài (850 từ), vô tình chứa các từ khóa nhạy cảm như "free", "call"

11.2. Nhóm False Negative (Để lọt 7 email Spam vào Hòm thư đến)

Danh sách 7 trường hợp False Negative trên tập Test

Source ID	Độ dài (từ thô)	Điểm xác suất (Prob)	Dấu hiệu nhận biết	Phân tích nguyên nhân
3168	12	0,5435	Spam tinh vi, không từ khóa lộ	Spam ngắn chào mời khéo léo, điểm xác suất đạt 0,5435 (sát dưới ngưỡng 0,555)
849	238	0,5096	Xác suất dưới ngưỡng cắt	Spam dạng bài viết quảng cáo dài, chèn văn bản ngụy trang làm loãng mật độ từ rác
3642	28	0,5362	Spam tinh vi, không từ khóa lộ	Spam ngắn không chứa từ khóa lộ liễu, điểm đạt 0,5362 (dưới ngưỡng 0,555)
2793	445	0,3983	Xác suất dưới ngưỡng cắt	Thư rác dài (445 từ) giả mạo bản tin tin tức, mô hình tuyến tính bị đánh lừa bởi từ vựng thông thường
3817	173	0,5334	Spam tinh vi, không từ khóa lộ	Thư mời hội thảo lừa đảo nhưng dùng văn phong học thuật chuẩn mực
4193	33	0,4610	Spam tinh vi, không từ khóa lộ	Chào mời dịch vụ không kèm liên kết trực tiếp, điểm xác suất chỉ đạt 0,4610
4840	923	0,5268	Xác suất dưới ngưỡng cắt	Spam rất dài (923 từ), nội dung kể chuyện giả mạo, làm giảm trọng số trung bình

> **Ghi chú quan sát thực tế:**
>
> - Không có mẫu lỗi nào có `source_id = 354` trong tập Test hiện tại.
> - Độ dài của các mẫu False Negative trải rộng từ **12 từ đến 923 từ**; quan sát trước đây cho rằng "mọi mẫu FN đều ngắn dưới 15 từ" là không đúng với dữ liệu thực nghiệm.
> - Biểu thức chính quy `URL_PATTERN` của bài lab nhận diện 0 liên kết URL trong toàn bộ 17 mẫu lỗi trên. Lý do là regex chỉ bắt chuỗi có tiền tố `http://`, `https://` hoặc `www.`, trong khi các email thô trong tập dữ liệu có thể đã bị ngắt dòng hoặc tách các dấu chấm của liên kết thành các token riêng biệt.

12. Đóng gói, tải nạp và dịch vụ dự đoán email mới

12.1. Đóng gói Artifact hoàn chỉnh (Bundle Serialization)

Toàn bộ trạng thái sau huấn luyện được lưu trữ trong tệp `models/spam_classifier_bundle.joblib` bằng thư viện Joblib. Gói bundle bao gồm:

- Mô hình đã huấn luyện: `Linear SVM (C=1.0)` với vector trọng số $w \in \mathbb{R}^{500}$ và bias b.
- Bộ trích xuất đặc trưng: `EmailFeatureBuilder` lưu từ vựng, vector IDF, thông số min/max scaler và ngưỡng cắt độ dài 791 từ.
- Danh sách 500 đặc trưng được chọn: Đảm bảo thứ tự và ý nghĩa của từng cột trong ma trận đặc trưng luôn cố định.
- Ngưỡng quyết định tối ưu: Giá trị $\tau = 0{,}555$ được chốt từ tập Validation.
- Metadata: Dấu thời gian, phiên bản thư viện, cấu hình huấn luyện.

12.2. Lớp dịch vụ dự đoán (SpamInferenceService)

Lớp `SpamInferenceService` trong `src/deployment/predictor.py` hỗ trợ nạp mô hình và dự đoán cho email mới bằng mã Python:

```
from src.deployment.predictor import SpamInferenceService

service = SpamInferenceService("models/spam_classifier_bundle.joblib")
result = service.predict_email("URGENT: Your account has been suspended! Click here to verify.")

print(result)
# {
#   "label": 1,
#   "prediction": "Spam",
#   "spam_probability": 0.8421,
#   "threshold": 0.555,
#   "detected_signals": ["urgent", "click", "exclamation"],
#   "suspected_cause": "Dính từ khóa nhạy cảm (urgent, click)"
# }
```

> **Trường `detected_signals` và `suspected_cause` trong kết quả có phải là giải thích mô hình (Feature Attribution) không?**
>
> Không. Trong bài lab, `detected_signals` và `suspected_cause` là **các luật mô tả kinh nghiệm (rule-based heuristics)** được viết thêm để giúp người dùng quan sát các biểu hiện bề mặt của email. Chúng không phản ánh mức độ đóng góp toán học thực tế của từng đặc trưng vào điểm số quyết định của Linear SVM (như các phương pháp SHAP hay LIME).

Phạm vi bài lab: Chương trình dừng lại ở mức cung cấp lớp Python cục bộ để nạp tệp bundle và chạy dự đoán; chưa xây dựng thành web service API (FastAPI/Flask) hay kết nối trực tiếp với hòm thư thực tế.

13. Kiểm thử tự động và khả năng tái lập (Verification)

13.1. Kết quả kiểm thử bộ mã nguồn

Bộ kiểm thử tự động của bài lab gồm **60 ca kiểm thử đơn vị (Unit Tests)** trong thư mục `tests/`, được chạy thông qua lệnh chuẩn của Python:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Kết quả kiểm tra: **60/60 tests pass thành công 100%** trong thời gian 21 giây. Bộ kiểm thử bao phủ các nội dung:

- Kiểm tra hàm nạp dữ liệu, hàm khử trùng lặp và phân tách dữ liệu phân tầng.
- Kiểm tra bộ trích xuất đặc trưng, tính toán TF-IDF, Min-Max scaler và cắt độ dài.
- Kiểm tra thuật toán LR, SVM, Naive Bayes về hội tụ gradient và tính toán ma trận thưa.
- Kiểm tra quá trình quét ngưỡng cắt và tính toán các chỉ số đánh giá.
- Kiểm tra hồi quy (Regression tests) bảo đảm dung sai gradient `tol` của Linear SVM, kiểm tra trọng số voting và cấu hình dữ liệu phân tích lỗi.

> **Lưu ý về kiểm thử:** Việc vượt qua 60 bài kiểm thử chứng minh chương trình chạy đúng theo các tình huống đã được viết trong bộ test; đây là công cụ hỗ trợ kiểm tra lỗi trong quá trình lập trình bài lab.

13.2. Hướng dẫn chạy lại toàn bộ bài lab

Để chạy lại toàn bộ quy trình từ dữ liệu thô đến việc tạo kết quả:

```
# 1. Kích hoạt môi trường ảo
.\.venv\Scripts\Activate.ps1

# 2. Chạy pipeline huấn luyện và đánh giá chính
python main.py

# 3. Chạy toàn bộ bài kiểm thử
python -m unittest discover -s tests -v
```

14. Giới hạn của bài lab và hướng phát triển tiếp theo

14.1. Các giới hạn của bài lab

1. **Kích thước tập dữ liệu:** Dữ liệu có quy mô khoảng 5.000 email từ tập Enron cũ. Dữ liệu này chưa phản ánh hết sự đa dạng và các hình thức thư rác mới hiện nay.
2. **Giới hạn của mô hình Bag-of-Words:** Việc biểu diễn văn bản dựa trên tần số từ đơn và cặp từ (n-gram) chưa nắm bắt được ngữ cảnh sâu và thứ tự câu từ phức tạp.
3. **Chưa hiệu chuẩn xác suất (Uncalibrated Probabilities):** Điểm số đầu ra của Linear SVM (qua hàm sigmoid) và Complement Naive Bayes chưa được hiệu chuẩn xác suất (như dùng Platt Scaling), nên điểm số chỉ thể hiện mức độ nghi ngờ tương đối chứ chưa phải xác suất thống kê chuẩn xác.
4. **Tiền xử lý cố định:** Danh sách từ dừng và biểu thức chính quy cố định có thể bỏ sót hoặc xử lý chưa linh hoạt với các biến thể văn bản mới.

14.2. Hướng phát triển tiếp theo

- Thử nghiệm kiểm định chéo phân tầng lặp lại (Repeated Stratified K-Fold) trên tập dữ liệu đa dạng hơn.
- Áp dụng hiệu chuẩn xác suất (Platt Scaling hoặc Isotonic Regression) cho Linear SVM.
- Tìm hiểu và thử nghiệm các mô hình ngôn ngữ dựa trên Transformer (như DistilBERT) để so sánh hiệu quả với các mô hình tuyến tính cổ điển.
- Xây dựng giao diện web đơn giản hoặc API bằng FastAPI để thử nghiệm nhập email trực tiếp.

15. Hồ sơ dữ liệu và mã nguồn đối chiếu của bài lab

Toàn bộ phân tích và số liệu trong báo cáo được lưu trữ và đối chiếu trực tiếp tại các tệp trong thư mục dự án:

- **Mã nguồn thực thi chính:** `main.py` và các module trong thư mục `src/` (data, features, models, evaluation, deployment).
- **Siêu dữ liệu lần chạy thực nghiệm:** `results/metrics/run_summary.json` (ghi nhận thời gian chạy 05/11/2026, thông số cấu hình và mã băm SHA-256 các tệp nguồn).
- **Số liệu kiểm tra dữ liệu:** `results/metrics/data_audit.json` (thống kê khử trùng lặp và số lượng mẫu từng tập).
- **Số liệu quét đặc trưng:** `results/metrics/top_k_sweep.csv` (kết quả các mức K từ 500 đến 4.318).
- **Số liệu so sánh mô hình:** `results/metrics/validation_model_selection.csv` và `results/metrics/test_models_comparison.csv`.
- **Dữ liệu mẫu phân loại sai:** `results/metrics/misclassified_samples.csv` (chi tiết 17 mẫu lỗi trên tập Test).
- **Tài liệu hướng dẫn của bài lab:** `README.md`.

Báo cáo thực hành môn Học máy – Practice 1: Phân loại email Spam và Ham. Cập nhật theo kết quả thực nghiệm ngày 05/11/2026.